

Evaluating Randomised Algorithms and Their LLM Generated Counterparts on the Bin Packing Problem

Joshua Raybould

March 2026

Abstract

The one-dimensional offline Bin Packing Problem (BPP) is a strongly NP-hard combinatorial optimisation problem with wide-ranging practical applications. This study addresses three research questions centred on the empirical performance of randomised algorithms for the BPP and the capability of Large Language Models (LLMs) to generate competitive implementations of those algorithms from scratch.

To establish a robust baseline, we implemented, tuned, and evaluated eight stochastic algorithms spanning simple heuristics (Randomised Best Fit), local search methods (Simulated Annealing, Tabu Search, Variable Neighbourhood Search), multi-start approaches (GRASP), population-based evolutionary methods (Grouping Genetic Algorithm), and swarm intelligence (Ant Colony Optimisation, Fitness Dependent Optimizer). All algorithms were benchmarked across custom-generated instance sets ranging from 100 to 1600 items, with solution quality measured against the L_2 lower bound of Martello and Toth. Among the human implementations, GGA, VNS, and SA formed a strong top tier, with GGA achieving the best overall solution quality and SA offering the most favourable balance of quality and computational efficiency due to its near-flat runtime scaling.

To evaluate LLM code generation capability, we developed an automated iterative refinement pipeline incorporating correctness validation and performance-driven optimisation, and applied it to two flagship models: OpenAI’s GPT-5.2 and Anthropic’s Claude Opus 4.6. Each model was tasked with generating implementations for all seven non-trivial algorithms under a zero-shot prompting strategy with Chain-of-Thought reasoning. The resulting implementations were evaluated against the human baselines on standard BP-PLIB benchmark instances, with each implementation allocated an identical wall-clock time budget.

The results show that both LLMs produced functionally correct code for six of the seven algorithms, deviating from the specified paradigm only on FDO, and frequently matched or exceeded human-level performance. Anthropic’s Claude Opus 4.6 achieved the lowest weighted average deviation from the known optimal on four of the seven algorithms, incorporating design choices absent from the human baselines—notably a dynamic cooling schedule for Simulated Annealing and an iterative bin-reduction formulation for Tabu Search. The human implementations retained an advantage on the Genetic Algorithm and Variable Neighbourhood Search, where the human’s choice of bin-level encoding and exhaustive local search respectively outperformed the alternative approaches adopted by both LLMs. OpenAI’s GPT-5.2 exhibited higher variance, achieving the strongest result on ACO but struggling on SA due to a fundamental objective function flaw that rendered the search almost entirely uninformative despite producing syntactically correct and executable code. Neither model produced a faithful adaptation of the Fitness Dependent

Optimizer, instead defaulting to more familiar algorithmic paradigms. These findings suggest that flagship LLMs are now capable of producing competitive metaheuristic implementations for well-studied combinatorial problems, but remain susceptible to subtle reasoning errors that are difficult to detect through automated correctness testing alone.

1 Introduction

1.1 Problem Background

The bin packing problem (BPP) is one of the central problems in combinatorial optimisation. In its standard formulation, a set of items with corresponding values (generally size or weight) must be packed into an unlimited supply of bins, each with a uniform capacity. The objective is to assign the items to the bins such that the total number of bins used is minimised. This is the one-dimensional (1D) bin packing problem, which is conceptually similar to the 1D cutting stock problem [1]. The 1D BPP is primarily studied in two variants: the offline problem, where the algorithm has access to the weights of all items simultaneously, and the online problem, where items are revealed and packed one at a time. In this study, we focus on the offline case.

The 1D BPP is classified as NP-Hard, meaning that no known polynomial-time algorithm exists for computing an exact solution. Consequently, heuristic and randomised algorithms are often employed to obtain near-optimal solutions efficiently. Beyond its theoretical significance, bin packing has numerous real-world applications, ranging from loading transport vehicles with weight capacities to minimising waste in industrial cutting processes [2]. Additionally, many complex scheduling problems contain the BPP as a special case.

1.2 Problem Definition

Here we formalise the classical offline 1D BPP. The following definition is relatively standard and is arrived at with reference to [1].

Let C denote the uniform capacity of each bin, and let w_i denote the weight (or equivalently, size) of item i , such that $w_i \leq C$ for all items. We define u as an upper bound on the minimum number of bins required to pack all n items. We introduce the binary decision variable y_j , where $y_j = 1$ if the j -th bin is opened (used in the solution), and 0 otherwise, for $1 \leq j \leq u$. Items are labeled $i \in \{1, \dots, n\}$, and we define a second binary variable x_{ij} , which equals 1 if item i is packed in bin j , and 0 otherwise. The objective function is:

$$\min \sum_{j=1}^u y_j$$

This aims to minimise the total number of open bins, subject to the following constraints:

$$\begin{aligned} \sum_{j=1}^u x_{ij} &= 1, \quad \forall i = 1, \dots, n \\ \sum_{i=1}^n x_{ij} w_i &\leq C y_j, \quad \forall j = 1, \dots, u \end{aligned}$$

The first constraint ensures that each item is packed into exactly one bin. The second constraint guarantees that the sum of the weights of the items in any given bin does not exceed the bin capacity C .

1.3 Complexity Analysis

The 1D offline BPP is strongly NP-hard, which can be established via a polynomial-time reduction from the 3-Partition problem [3]. Given an instance of 3-Partition consisting of $3m$ integers $a_1, \dots, a_{3m}$ where $B/4 < a_i < B/2$ for all i and $\sum_{i=1}^{3m} a_i = mB$, we construct a BPP instance with bin capacity B and items of exactly those weights. A valid packing into m bins exists if and only if a solution to 3-Partition exists. Since each $a_i > B/4$, no bin can hold more than three items, and since each $a_i < B/2$, no bin can hold fewer than two; every bin must therefore contain exactly three items summing to B . Because 3-Partition is strongly NP-complete [3], BPP is strongly NP-hard. As a consequence, no polynomial-time algorithm for solving the BPP to optimality is known, assuming $P \neq NP$. For large-scale instances, approximation algorithms and stochastic metaheuristics that trade guaranteed optimality for computational efficiency are therefore necessary.

1.4 Randomised Algorithms

Randomised algorithms rely on random variable generation to influence their execution paths, making them non-deterministic. In this context, we define stochastic optimisation as the use of randomness to navigate a search space and find near-optimal solutions efficiently [4]. While strict definitions of stochastic optimisation often require a noisy objective function, the algorithms implemented here primarily use random choices to direct the search process. Therefore, these algorithms largely belong to the subfield of metaheuristics, which are problem-independent frameworks that guide and modify underlying heuristics to escape local optima [5].

These algorithms vary drastically in complexity. The simplest approaches might involve randomising the input order before applying a naive greedy heuristic, which is particularly useful for mitigating worst-case inputs generated by adversaries. Conversely, more complex metaheuristics draw inspiration from biological evolution and natural systems. A primary goal of this paper is to benchmark a diverse variety of these approaches.

1.5 Large Language Models (LLMs)

LLMs are trained on massive corpora of text and code, enabling them to capture semantic meaning and structural syntax across both natural and programming languages. This has made them highly capable of generating code, as programming languages share structural similarities with natural text.

Naturally, there are limitations. The code produced by LLMs is not guaranteed to be correct and requires rigorous verification. While models perform exceptionally well on tasks heavily represented in their training data, they often lack the capacity for deep reasoning in novel scenarios [6]. Consequently, for common tasks (e.g., the Mostly Basic Python Programming dataset), modern LLMs generate reliable solutions. Furthermore, while metaheuristics for the

BPP are not an entirely niche domain, the empirical nature of algorithm design poses unique challenges. Specifically, selecting optimal hyper-parameters—such as cooling schedules in Simulated Annealing—and managing the complex interplay between different algorithmic components typically require extensive trial-and-error and human intuition. As these nuances are likely difficult to deduce from the static code found in training corpora, it remains unclear how capable LLMs are at producing performant metaheuristics.

1.6 Research Questions

This project aims to compare the empirical performance of human-written randomised algorithms for the 1D offline BPP against implementations generated by state-of-the-art LLMs. Specifically, our key research questions are:

1. How do various human-implemented randomised algorithms for the BPP compare in empirical performance across diverse problem instances? This establishes a robust baseline and provides insight into practical algorithmic efficiency.
2. How effective are flagship LLMs at producing functionally correct metaheuristic implementations that achieve competitive solution quality within a fixed time budget?
3. To what degree does performance vary across different LLMs, and are there identifiable patterns or common algorithmic shortcomings in their generated code?

2 Literature Review

2.1 Traditional Algorithms for the 1D BPP

The 1D BPP has been studied extensively for over five decades, producing a rich landscape of exact, heuristic, and metaheuristic approaches. Exact methods, including branch-and-bound and column generation, can solve instances to optimality but become computationally intractable as instance size grows, reflecting the problem’s NP-hardness [3, 1]. For large-scale instances, practitioners therefore rely on approximation algorithms and metaheuristics that trade guaranteed optimality for computational efficiency [2].

Widely used constructive heuristics include First Fit (FF), Best Fit (BF), and their decreasing variants (FFD and BFD), which sort items by decreasing weight prior to packing. FFD is particularly well-regarded, admitting a tight worst-case guarantee of $\frac{11}{9}\text{OPT} + \frac{6}{9}$ [7], and is commonly used as an initialization strategy for more sophisticated methods. Beyond these, a broad range of metaheuristics has been applied to the BPP. Simulated Annealing approaches have been explored by Sonuç et al. [8], who use FFD initialization and a sum-of-squares objective to guide the search. Variable Neighbourhood Search was explored by Fleszar and Hindi [9], whose VNS implementation improves upon the Minimum Bin Slack heuristic in approximately 23% of cases. The Grouping Genetic Algorithm, introduced by Falkenauer [10] and extended by Quiroz-Castellanos et al. [11] in the GGA-CGT framework, addresses the failure of standard genetic operators on grouping problems by encoding bins rather than items as the unit of evolution. Ant Colony Optimisation has been adapted for the BPP by

Levine and Ducatelle [12], who assign pheromone between item weights, allowing the colony to learn which weight combinations pack efficiently together.

While numerous studies compare subsets of these algorithms, comprehensive cross-paradigm empirical comparisons remain rare [2]. Existing surveys tend to focus on algorithmic novelty or theoretical guarantees rather than standardised empirical performance across diverse instance classes. This makes it difficult for practitioners to assess the relative practical effectiveness of different paradigms, and motivates the baseline evaluation conducted in Section 5 of this study.

2.2 Large Language Models in Code Generation

LLMs are trained on massive corpora of text and code, enabling them to capture semantic meaning and structural syntax across both natural and programming languages [13]. This has made them highly capable of generating code, as programming languages share structural regularities with natural text. Experiments with models such as GPT-4 have shown that LLMs can successfully write greedy algorithms and solve dynamic programming problems, occasionally surpassing capable university students in these domains [14, 13]. On standard evaluation datasets such as HumanEval and the Mostly Basic Python Programming benchmark, modern LLMs generate reliable solutions for well-represented tasks. However, LLMs are known to struggle with tasks that require genuine reasoning in novel or underrepresented scenarios, where pattern-matched knowledge from training data is insufficient [6].

2.3 Evaluating LLM Performance on Complex Algorithmic Tasks

To quantify generative capability beyond standard benchmarks, multiple studies have compared human-written code to LLM-generated solutions for competitive programming challenges such as those found on LeetCode [14, 15, 16]. These problems exhibit higher logical complexity than standard evaluation datasets, providing a closer proxy to the difficulty of implementing BPP metaheuristics.

The findings of Coignon et al. [16] are particularly instructive. In terms of execution speed, LLMs frequently outperformed human developers, beating 73% of prior submissions on average. Functional correctness was reasonable for well-represented problems, averaging approximately 38% on the most popular LeetCode questions. However, LLM performance proved highly sensitive to task novelty: on newly released questions absent from training corpora, correctness deteriorated sharply, with the strongest model achieving only 9.5% on a single-attempt basis. This suggests that LLM code generation performance conflates knowledge retrieval with genuine algorithmic synthesis, a distinction that becomes critical when evaluating performance on tasks where subtle design decisions — such as objective function selection and parameter tuning — have an outsized impact on solution quality.

2.4 LLMs in Metaheuristic Design and Code Generation for Combinatorial Problems

A growing body of work has begun to examine the use of LLMs for automating algorithm design in combinatorial optimisation. A prominent strand of this research focuses on using

LLMs to generate novel heuristics from scratch. ReEvo [17] introduces Language Hyper-Heuristics, combining evolutionary search with LLM-generated verbal reflections to automatically produce and refine heuristic code across multiple combinatorial problems, frequently matching or surpassing human-designed approaches. This line of work is primarily concerned with algorithm innovation.

A complementary direction uses LLMs to generate complete search algorithms for specific combinatorial problems. Rosin [18] develops a protocol in which LLMs freely select and implement heuristic strategies — including simulated annealing and genetic algorithms — for combinatorial design problems, using automated execution feedback from a validity verifier to evaluate and select among a large pool of candidate implementations. This work demonstrates that LLM-generated code can resolve previously open mathematical instances.

The present study contributes to this landscape by evaluating the ability of LLMs to implement a specified set of established metaheuristic paradigms from scratch, directly comparing the resulting implementations against carefully tuned human-written baselines on a well-studied combinatorial optimisation problem. Rather than allowing LLMs to freely design or discover algorithms, this framing separates two questions that are often conflated: whether LLMs can produce performant code, and whether they can faithfully implement a given algorithmic paradigm — including making appropriate design decisions such as objective function selection and parameter selection.

3 Human-Implemented Metaheuristics

To establish a robust empirical baseline, we implemented a diverse suite of stochastic algorithms for the 1D BPP. Parameter values were selected through manual trial-and-error tuning on a set of uniform instances generated using the same methodology as our main evaluation dataset (described in Section 4), hereafter referred to as test-u. This dataset consists of 10 instances of 400 items and 10 instances of 800 items. A parameter configuration was considered valid if the average execution time across two independent runs rounded to 40 seconds or less. Among valid configurations, we selected the one achieving the lowest average percentage difference from the theoretical lower bound. Where multiple configurations produced similar solution quality, we favoured the one expected to generalise better across varying instance sizes. The resulting parameters target a maximum average execution time of approximately 20 seconds per run, keeping algorithms with steep runtime scaling tractable at the larger instance sizes evaluated in Section 5, where no time limits are imposed. All algorithms were designed, written, and tuned by the author with reference to the relevant literature, with LLMs consulted solely to identify bugs and oversights. Corrections were adopted only where the fix was consistent with the original implementation intent, with all substantive design decisions remaining entirely human-driven.

3.1 Common Algorithmic Components

3.1.1 Lower Bounds

We first establish the theoretical lower bounds utilised as a strict stopping criterion for the evaluated algorithms. Employing a tight lower bound ensures that algorithms do not waste

computational time searching once an optimal solution has been achieved. A simple, well-known lower bound on the number of bins, computable in $O(n)$ time, is given by [19]:

$$L_1 = \left\lceil \frac{\sum_{i=1}^n w_i}{C} \right\rceil$$

This bound is derived by considering an LP-relaxation of the problem, wherein items are allowed to be fractionally split across bins. In the optimal solution for this relaxation, every open bin is filled entirely to capacity, except potentially the last, which contains the residual weight [20]. Despite its simplicity, L_1 is highly effective when item weights are small relative to the capacity, guaranteeing a worst-case bound of $\frac{1}{2}$ the optimal solution value [19, 20]. However, because the problem instances evaluated in this study feature larger item weights, L_1 is often too loose to trigger early termination.

We instead employ the L_2 bound developed by Martello and Toth [19], which guarantees at worst $\frac{2}{3}$ of the optimal solution value. It requires $O(n \log n)$ time, which is dominated by the initial need to sort the items by decreasing weight. We briefly outline the bound here; for a comprehensive derivation, we direct the reader to [19]. For any integer K such that $0 \leq K \leq \frac{1}{2}C$, the items are partitioned into three sets:

$$\begin{aligned} I_1 &= \{i \in \{1, \dots, n\} \mid w_i > C - K\}, \\ I_2 &= \{i \in \{1, \dots, n\} \mid C - K \geq w_i > \tfrac{1}{2}C\}, \\ I_3 &= \{i \in \{1, \dots, n\} \mid \tfrac{1}{2}C \geq w_i \geq K\}. \end{aligned}$$

Martello and Toth then define the lower bound for a given K as:

$$L_2(K) = |I_1| + |I_2| + \max \left\{ 0, \left\lceil \frac{\sum_{i \in I_3} w_i - (|I_2|C - \sum_{i \in I_2} w_i)}{C} \right\rceil \right\}$$

This represents a valid bound for every possible value of K . The first two terms account for items in I_1 and I_2 , which are strictly greater than half the bin capacity and therefore cannot share a bin with one another. The third term calculates the additional bins required if the total weight of the items in I_3 exceeds the combined residual space left in the I_2 bins.

Because the tightness of the bound is dependent on K , the strongest lower bound is obtained by maximizing $L_2(K)$ across all valid K . To achieve the desired $O(n \log n)$ runtime, the item weights are first sorted in decreasing order, and K is restricted to values that explicitly appear in the item weights. Furthermore, because any item in I_1 or I_2 is strictly greater than $\frac{1}{2}C$, the combined cardinality $|I_1| + |I_2|$ is invariant with respect to K . Our implementation precomputes this constant size once. Then, as we iterate through valid values of K , we incrementally update the sums $\sum_{i \in I_2} w_i$ and $\sum_{i \in I_3} w_i$, incrementing the size of $|I_2|$ only as required for the third term.

Note that while L_2 is the bound we use, there is a slightly tighter bound which makes use of dual feasible functions, as developed by Fekete and Schepers [20].

3.1.2 Fitness Evaluation

If the objective relied solely on the total number of active bins, it would remain unchanged for the vast majority of transitions — since a move only affects the bin count when it completely empties a bin — leaving the search with no signal to distinguish between neighbouring solutions [10].

To circumvent this, we utilise the sum-of-squares objective function, a widely adopted approach originally introduced by Falkenauer for evolutionary bin packing [10] and subsequently employed across a range of metaheuristic implementations [8, 9]. We define the fitness of a candidate solution as:

$$f = \sum_{j=1}^u \left(\sum_{i=1}^n x_{ij} w_i \right)^2$$

This yields a higher fitness score when items are concentrated into tightly packed bins rather than distributed evenly across multiple bins. This formulation guides the algorithm to starve lightly packed bins until they can be emptied completely, thereby reducing the total bin count.

3.2 Best Fit With Randomised Order

3.2.1 Overview

Best Fit is a classic heuristic typically applied to the online version of the bin packing problem. While rarely used in isolation for the offline variant, where stronger alternatives exist, it provides the simplest possible randomised baseline for comparison. The algorithm operates identically to the standard Best Fit, placing each item into the bin with the tightest available space, but randomises the order of the input items before execution. The purpose of this is to mitigate the impact of adversarial or worst-case input orderings, allowing the heuristic to generally achieve better packings [21]. We refer to this combined procedure as Randomised Best Fit (RBF).

3.2.2 Implementation

The algorithm first randomly shuffles the input items, then applies the standard Best Fit heuristic sequentially. For every item, the algorithm scans all currently open bins and places it into the bin that minimises the residual capacity. If an item cannot fit into any existing bin, a new bin is opened. This process continues until all items have been packed.

3.3 Simulated Annealing (SA)

3.3.1 Overview

Simulated Annealing (SA) is a probabilistic metaheuristic closely related to hill-climbing. A standard hill-climbing algorithm iteratively generates neighboring candidate solutions and

transitions to them only if they offer an improvement. SA modifies this approach by introducing a probability of transitioning to a worse solution. This mechanism provides the search trajectory with opportunities to escape local optima, significantly increasing the likelihood of converging on the global optimum. The probability of accepting a deleterious move depends on a temperature value; the higher this is the more likely the algorithm is to move to worse solutions. The temperature decays over time according to a predefined cooling schedule [4], [22].

3.3.2 Implementation

The algorithm generates its initial candidate solution using the First-Fit Decreasing (FFD) heuristic, mirroring the initialization strategy of Sonuç et al. [8]. Candidate solution quality is evaluated using the sum-of-squares objective function previously defined, with the incremental score change normalised by the bin capacity to ensure scale-invariance.

To transition to a neighbouring solution, the algorithm selects two distinct bins at random. It then has an equal probability of either moving a single random item from the first bin to the second, or swapping two random items between them. Infeasible moves, those that would violate a bin’s capacity constraint, are rejected outright, with no temperature update or acceptance calculation performed.

We initialize the temperature $T = 20,000$ with an exponential cooling schedule of $\alpha = 0.9996$. The algorithm terminates when $T \leq 0.01$, and has no fixed iteration limit.

3.4 Tabu Search (TS)

3.4.1 Overview

Tabu Search (TS) is a metaheuristic designed to overcome the primary limitation of standard local search methods: getting trapped in local optima. It functions similarly to hill-climbing but incorporates a short-term memory structure known as a tabu list. This list records recently visited configurations, forbidding the algorithm from returning to them for a set number of iterations. By making recent moves tabu, the algorithm is forced to explore novel regions of the search space, allowing it to escape local optima and discover stronger global configurations [4, 22].

3.4.2 Implementation

Similar to our Simulated Annealing approach, the TS algorithm generates an initial candidate solution using the First-Fit Decreasing (FFD) heuristic and evaluates fitness using the sum-of-squares objective function.

During each iteration, the algorithm explores the localised neighbourhood by generating 35 candidate moves. A move consists of selecting two random bins and either swapping two items between them or moving a single item from one to the other. To aggressively reduce the total bin count, we implement a targeted heuristic inspired by Lodi et al. [23]: when moving a single item, there is a 60% probability that the algorithm will deliberately select

the least full bin as the source. Because the emptiest bins are the easiest to empty entirely, this actively guides the search toward configurations that utilise fewer bins.

The algorithm executes for $I = 18,500$ iterations, selecting the best valid neighbour in each step even if it results in a lower fitness score than the current state. To prevent cycling without incurring excessive computational overhead, we maintain a tabu list recording the 4 most recent moves, storing the configurations of both affected bins for each. Finally, we implement a standard aspiration criterion: the tabu status of a move is completely ignored if executing that move directly results in the elimination of a bin, as this guarantees a strict improvement to the global objective.

3.5 Variable Neighbourhood Search (VNS)

3.5.1 Overview

Introduced by Mladenović and Hansen [24], VNS is a metaheuristic that works by systematically changing neighbourhoods during a perturbation phase (shaking) followed by a local search (often called the descent phase) [22].

The algorithm requires defining a set of neighbourhood structures, N_k , for $1 \leq k \leq k_{\max}$, where $k_{\max}$ is a user-specified parameter. A neighbourhood N_k could, for example, be defined as being k moves away from the current solution. In each iteration of the algorithm, we start with $k = 1$ and continue until $k = k_{\max}$. While $k \leq k_{\max}$, we obtain a new candidate solution by "shaking" the incumbent solution, moving from the incumbent to a random solution within its k -th neighbourhood. We then apply a local search to this new candidate solution. This is often executed using best improvement (which takes the direction of steepest descent) or first improvement (which selects the first discovered move that improves the solution).

After the local search, the resulting solution is evaluated using the objective function. If this new solution is strictly better than the incumbent, it becomes the new incumbent, and k is reset to 1. Otherwise, k is incremented to explore a wider neighbourhood. By systematically exploring larger neighbourhoods, the algorithm significantly increases its probability of escaping the current local optimum. As noted in [22], local minima with respect to one or several N_k are relatively close to one another, implying that local optima tend to contain structural information about the global optimum.

3.5.2 Implementation

We generate our initial incumbent solution using the FFD heuristic. It should be noted that much of our implementation is inspired by the approach of Fleszar and Hindi [9], who utilise a different heuristic called Minimum Bin Slack (MBS) that generally outperforms FFD [25]. Fleszar and Hindi's VNS algorithm improves the solution arrived at by their MBS' heuristic in approximately 23% of cases. We chose to utilise FFD instead of MBS for the sake of time and also to maintain a more consistent initialisation baseline across our algorithms, ensuring that any performance differences observed are attributable to the VNS mechanism rather than the starting heuristic. Additionally, FFD is more widely used.

Following initialisation, we measure solution fitness using the sum-of-squares objective func-

tion. To optimise the local search phase, we sort our items by weight in non-decreasing order. During the shaking phase, we maintain a list of items that have not yet been moved. The algorithm selects an unmoved item at random and attempts to move it (either by swapping it with another unmoved item or transferring it to a different bin, provided capacity constraints are satisfied). Once an item is involved in a move, it is removed from the eligible list for the remainder of that shaking step.

After shaking, we execute a best-improvement local search by systematically evaluating every possible move across the entire solution space. Specifically, for each eligible item (excluding those in completely full bins), the algorithm evaluates transferring it to every other open bin, as well as swapping it with every other eligible item. The move that yields the greatest improvement is selected. To optimise execution, the Δ fitness improvement is calculated by considering only the bins where weights actually change, as the squared sum of all other bins remains constant.

Because the items are processed in non-decreasing order of weight, we can speed up the search. When iterating through items to transfer into a specific bin, the inner loop terminates immediately upon encountering an item whose weight exceeds the available residual space.

The algorithm terminates when the theoretical lower bound is reached or after a strict limit of 4 iterations. We set $k_{\max} = 3$; although this is relatively low, the exhaustive best-improvement local search is computationally expensive, and empirical testing demonstrated that larger neighbourhoods incurred unreasonable computational costs without a commensurate improvement in solution quality.

3.6 Greedy Randomised Adaptive Search Procedure (GRASP)

3.6.1 Overview

GRASP is a multi-start metaheuristic consisting of two main phases: a constructive phase and a local search phase [22]. These two phases are executed iteratively, and the highest-quality solution found across all iterations is ultimately returned.

The construction phase generates a candidate solution by iteratively adding components. Rather than acting purely greedily, it selects components at random from a Restricted Candidate List (RCL)—a subset of the remaining components that yield the lowest cost or highest quality. The RCL is typically defined by a threshold quality. For example, if $q^{\min}$ and $q^{\max}$ denote the qualities of the worst and best available components respectively, the threshold is defined as $q^{\min} + (1 - \alpha)(q^{\max} - q^{\min})$, where $\alpha \in [0, 1]$ controls the greediness of the search. An α value of 0 results in a purely greedy construction, while an α value of 1 results in a completely random construction.

Various enhancements can be applied to the standard GRASP framework. In Reactive GRASP, the parameter α is not fixed. Instead, the algorithm initializes with a discrete set of possible α values, each with an equal probability of selection. As the search progresses, these probabilities are periodically updated based on the historical performance of each α value. This feedback loop forces the algorithm to learn from previous iterations and bias the search toward parameters that yield superior solutions. Another notable enhancement is path-relinking, an intensification strategy that explores the trajectory connecting two high-quality solutions found during the search to discover potentially superior intermediate

configurations.

3.6.2 Implementation

We implemented a Reactive GRASP metaheuristic which runs for 10 iterations. During the constructive phase, the quality of an item is defined purely by its weight (size). Items are sequentially packed into the currently open bin. A new bin is only opened if no unassigned item can fit within the residual capacity of the active bin; otherwise, the algorithm restricts its selection exclusively to items that can successfully fit.

To govern the RCL, we define a candidate set of α values: $\{0.05, 0.10, 0.15\}$. These relatively small values heavily bias the construction toward pure greediness, which is highly effective given that item weight is a strong heuristic for the Bin Packing Problem (empirical testing revealed that $\alpha = 0.05$ consistently yielded the best performance). Initially, each candidate value has an equal probability of selection. We track the average cost (defined as the total number of bins used) of the solutions generated by each α .

Every two iterations, provided every α has been utilised at least once, the selection probabilities are updated. Let $\{\alpha_1, \alpha_2, \dots, \alpha_m\}$ denote the set of candidate values. The selection probability $P(i)$ for the i -th value, α_i , is calculated as:

$$Q(i) = \frac{N}{\text{avgcost}(i) - \ell \cdot \min_{1 \leq j \leq m} \text{avgcost}(j)}$$

$$P(i) = \frac{Q(i)}{\sum_{j=1}^m Q(j)}$$

where $Q(i)$ represents the relative performance quality of α_i , and $\text{avgcost}(i)$ is the historical average cost of solutions generated using α_i . The numerator N represents the total number of items being packed, included as a scaling factor for precision.

The parameter ℓ serves as a learning rate ($0 \leq \ell < 1$) that dictates how aggressively the probabilities are altered. Because the absolute difference in average bin counts between α values is often exceedingly small, ℓ is necessary to mathematically amplify these fractional differences. We set $\ell = 0.995$, allowing the probabilities to evolve organically over time without permitting any single α value to become overwhelmingly dominant, which would prematurely stifle exploration of the search space. Following the construction of a candidate solution, we execute the descent phase. For this, we directly utilise the Tabu Search metaheuristic defined previously. To balance intensification with computational efficiency, the inner Tabu Search is given 1,800 iterations per GRASP cycle.

3.7 Genetic Algorithm (GA)

3.7.1 Overview

The Genetic Algorithm (GA) is a population-based evolutionary metaheuristic inspired by natural selection. A standard GA maintains a population of candidate solutions (chromosomes) and iteratively applies breeding, crossover, and mutation operators to evolve the population toward an optimal solution.

However, as famously noted by Falkenauer [10], standard GAs perform exceptionally poorly on grouping problems like the 1D Bin Packing Problem. In a standard GA, the chromosome encodes information at the item level (e.g., which bin an item belongs to). When standard crossover operators are applied to these item-oriented strings, they completely destroy the carefully optimized structures of well-packed bins, resulting in highly unfit children.

To resolve this, Falkenauer introduced the Grouping Genetic Algorithm (GGA). The core innovation of the GGA is a paradigm shift in the encoding scheme: restructuring the chromosome into group and item parts, where genetic operators act primarily on the group part. This ensures that when crossover and mutation operators are applied, they manipulate, transfer, or delete entire, intact bins rather than scattering individual items.

3.7.2 Implementation

While our foundational encoding follows Falkenauer, the majority of our genetic operators and heuristics are directly based on the state-of-the-art GGA-CGT (Controlled Gene Transmission) framework proposed by Quiroz-Castellanos et al. [11]. We maintain a population of 16 chromosomes over a strict limit of 90 generations, with 3 elites preserved each generation. Fitness is evaluated using the sum-of-squares objective function. To facilitate efficient mutation, bins within each chromosome are maintained in descending order of fullness throughout the algorithm.

To initialise the population, we employ the specialised First-Fit initialisation strategy detailed in the GGA-CGT framework. The algorithm first identifies all items with a weight strictly greater than half of the bin capacity. Because these items can never share a bin with one another, they are immediately placed into their own distinct bins. The remaining items are then randomised and packed using the standard First-Fit heuristic. This guarantees a strong, structurally sound starting population.

To transition between generations, we utilise a standard elitist survival strategy: the 3 highest-fitness individuals are preserved intact and carried directly into the next generation. Six children are then generated via the parent selection and crossover mechanism described below. The remaining 7 slots in the population are filled by applying mutation directly to the solutions immediately following the elite set in the fitness ranking.

To generate the 6 children, we implement a parent selection mechanism inspired by the Controlled Gene Transmission (CGT) strategy [11]. For each required child, a parent pair is formed by drawing one parent randomly from the elite set and the second parent randomly from the non-elite set. This approach guarantees that high-quality genetic material from the elites is consistently transmitted to the offspring, while the non-elite parents inject necessary structural diversity to prevent premature convergence.

Crossover is applied with a 40% probability. Our crossover operator sorts the bins within both parents in descending order of fullness. It then iterates through the parents pairwise, and for each corresponding pair of bins inherits them in order of fullness — the fuller bin first, then the other. A bin is skipped entirely if inheriting it would introduce a duplicate item already present in a previously inherited bin.

Whether a child is generated via crossover or cloned from a parent, it is subjected to mutation. Our mutation operator deletes three bins from the chromosome: two selected at random, and

one targeted as the least full bin in the solution. As bins are maintained in descending order of fullness, this is always the final bin in the chromosome. This biases the search toward configurations that utilise fewer bins.

Following both crossover and mutation, chromosomes are frequently left with unassigned (free) items. To repair these solutions, we apply the rearrangement by pairs procedure from the GGA-CGT framework [11]. This procedure operates in two distinct stages. First, it iterates through a randomised permutation of the active bins and attempts to improve their packing density by swapping pairs of packed items with free items. Specifically, it searches for opportunities to replace two packed items with either a single heavier free item, or a pair of free items that fills the bin more effectively without violating capacity constraints. Any packed items removed during this process are added to the pool of free items. In the second stage, all remaining free items are reinserted into the solution using the First-Fit heuristic. To encourage optimal bin filling, these items are sorted in descending order of weight if the largest free item exceeds half the bin capacity; otherwise, they are processed in a random order.

3.8 Ant Colony Optimisation (ACO)

3.8.1 Overview

Ant Colony Optimisation (ACO) is a population-based metaheuristic inspired by the foraging behaviour of real ants. While initially moving randomly, ants leave behind chemical pheromone trails. Over time, ants travelling along shorter paths return to the nest faster, causing pheromone to accumulate more rapidly on those routes. Subsequent ants probabilistically favour paths with higher pheromone concentrations, creating a positive feedback loop that eventually guides the entire colony toward the shortest path.

First introduced as Ant System (AS), the framework has seen significant evolution through variants like Elitist AS and MAX-MIN Ant System (MMAS) [26], making it highly competitive for complex combinatorial problems [12].

In ACO, artificial ants iteratively construct solutions by selecting components. The probability $p_k(i, j)$ of an ant k transitioning from state i to component j is governed by the following standard equation [27], [22]:

$$p_k(i, j) = \begin{cases} \frac{[\tau(i, j)]^\alpha [\eta(i, j)]^\beta}{\sum_{c \in C_k(i)} [\tau(i, c)]^\alpha [\eta(i, c)]^\beta} & \text{if } j \in C_k(i), \\ 0 & \text{otherwise.} \end{cases}$$

Here, $\tau(i, j)$ represents the accumulated pheromone (the learned historical quality of the component), and $\eta(i, j)$ represents a heuristic value. The parameters α and β control the relative influence of the pheromone and the heuristic, respectively. $C_k(i)$ represents the set of feasible components that ant k can currently add to its solution. Once all ants have constructed complete solutions, the pheromone trails are updated: pheromone is added based on the quality of the solutions found, and a global evaporation parameter, ρ , is applied to progressively forget older, suboptimal paths and prevent premature convergence.

3.8.2 Implementation

Our implementation was heavily guided by Levine [12]. To mitigate computational overhead and maintain feasible runtimes, we utilise a highly constrained search space consisting of just 3 ants executed over 7 iterations. Through empirical testing and parameter tuning, we observed that this shortened execution window necessitated a heavier reliance on greedy choices over historical pheromone accumulation. Consequently, we set the heuristic importance quite high at $\beta = 10$, with the pheromone importance at $\alpha = 2$ and the evaporation rate at $\rho = 0.2$. The heuristic value η for an item is defined simply as its weight.

To calculate the aggregate pheromone value for a candidate item j being considered for an active bin b , we use the adaptation from Levine:

$$\tau_b(j) = \begin{cases} \sum_{i \in b} \tau(i, j) & \text{if } b \neq \emptyset \\ 1 & \text{otherwise.} \end{cases}$$

This dictates that if item j 's weight historically pairs well with the weights already present in bin b , it will have a higher probability of selection. If the active bin is empty (i.e., a new bin has just been opened), the pheromone score evaluates to 1, making the initial selection heavily dependent on the heuristic η .

When a solution is completed, the pheromone matrix is updated by first multiplying existing values by ρ to simulate evaporation. Pheromone is then deposited based on the fitness of the solution, calculated using the normalised sum-of-squares objective function established by Falkenauer [10]. The deposited amount is the product of the solution's fitness and the frequency of each weight pair within that solution.

To aggressively refine the search space, we incorporate three core concepts from the MAX-MIN Ant System (MMAS) introduced by Stützle and Hoos [26] (as Levine did):

1. **Iteration-Best and Global-Best Updates:** In each standard iteration, only the iteration-best ant is permitted to deposit pheromone. Furthermore, every γ iterations, the global-best ant updates the matrix. We set $\gamma = \lceil 500/n \rceil$, a threshold designed to control the balance between exploration and exploitation within our condensed iteration limit.
2. **Pheromone Trail Limits:** To prevent stagnation, pheromone values that fall below $\tau_{\min}$ are reset to the boundary. Following Levine [12], no upper bound is enforced on pheromone values, as weight pairs can appear multiple times within a single solution, making a strict $\tau_{\max}$ inapplicable. Instead, $\tau_{\max}$ is used solely to set the optimistic initial pheromone values (see below). Adapting from [26], we calculate these values as:

$$\tau_{\max} = \frac{1}{1 - \rho}$$

$$\tau_{\min} = \frac{\tau_{\max} (1 - \sqrt[p_{best}]{p_{best}})}{(avg - 1) \sqrt[p_{best}]{p_{best}}}$$

where p_{best} is the probability of constructing the best solution after the trails have converged, set here to 0.05, and avg is $n/2$. It is worth noting these are approximations in the context of the BPP, as the order of items placed within a bin does not alter the final configuration, slightly skewing the mathematical assumptions underpinning the original derivation.

3. Optimistic Initialization: To encourage broad early exploration, all initial pheromone values are optimistically set to $\tau_{\max}$.

3.9 Fitness Dependent Optimizer (FDO)

3.9.1 Overview

Introduced in 2019 by Abdullah and Ahmed [28], the Fitness Dependent Optimizer (FDO) is a population-based swarm intelligence metaheuristic designed for continuous search spaces. While it shares foundational mathematical similarities with Particle Swarm Optimization (PSO) — specifically the concept of updating a search agent’s position by adding a velocity or pace vector — FDO calculates this *pace* entirely differently.

Biologically, FDO is inspired by the reproductive swarming process of honey bees, specifically the behaviour of scout bees searching for a new hive. When a colony swarms, scout bees explore the environment to identify suitable targets. Once high-quality candidate hives are found, the scouts communicate this information to the swarm. When a consensus threshold is reached, the entire swarm navigates toward the best-identified location.

In the context of FDO, candidate solutions represent the scout bees, and the globally optimal solution represents the target hive. The algorithm initialises a population of scout bees that randomly explore the search space. During each iteration t , the position of a search agent X_i is updated according to:

$$X_{i,t+1} = X_{i,t} + \textit{pace}$$

The *pace* variable dictates the direction and magnitude of the agent’s movement and is governed by a fitness weight factor fw . For minimisation problems, this is defined as [28]:

$$fw = \left| \frac{f(X_{i,t}^*)}{f(X_{i,t})} \right| \times wf$$

where $f(X_{i,t})$ is the fitness of the current agent, $f(X_{i,t}^*)$ is the fitness of the global best solution discovered so far, and $wf \in \{0, 1\}$ is a user-defined weight factor controlling convergence behaviour. When $wf = 1$ the algorithm exhibits a high level of convergence and a low chance of coverage, whereas $wf = 0$ effectively removes the fitness weight from the equation entirely, providing a more stable search. Since $f(X_{i,t}^*) \leq f(X_{i,t})$ for minimisation, fw is bounded in $[0, 1]$, reaching 1 when the current solution matches the global best.

The value of fw then governs how *pace* is computed via a branching mechanism. When $fw \in \{0, 1\}$, the agent has either converged to the global best or the fitness weight has been disabled, and *pace* is computed as $X_{i,t} \times r$ for a random number $r \in [-1, 1]$, producing purely random exploration. When $0 < fw < 1$, the agent’s movement is directed relative to the global best solution. A randomly generated $r \in [-1, 1]$ then determines the direction of movement: if $r \geq 0$, the agent moves toward the global best according to $(X_{i,t} - X_{i,t}^*) \times fw$, whereas if $r < 0$, it moves in the opposite direction via $(X_{i,t} - X_{i,t}^*) \times fw \times (-1)$, introducing diversification to prevent premature convergence. In both directed cases, the magnitude of the step is scaled by fw , meaning agents with solutions closer to the global best take proportionally larger steps.

3.9.2 Implementation

Our implementation adapts the FDO framework for the discrete 1D Bin Packing Problem, drawing heavily upon the approach proposed by Abdul-Minaam et al. [29], who first adapted FDO for this domain. Whereas Abdul-Minaam et al. employ a minimisation objective, we evaluate fitness using the sum-of-squares objective function, maximised consistently across all other algorithms in this study. We therefore use the inverse of the minimisation formulation for fw , which preserves $fw \in [0, 1]$:

$$fw = \left| \frac{f(X)}{f(X^*)} \right| \times wf$$

where $f(X)$ is the fitness of the current solution and $f(X^*)$ is the fitness of the global best solution discovered so far. We set $wf = 1$.

We initialise a population of 20 candidate solutions using the randomised First-Fit heuristic and maintain a strict execution limit of 70 iterations, terminating early only if the theoretical lower bound is achieved.

At the beginning of each iteration, we identify both the global best solution (X^*) and the worst solution in the current population. To introduce a strong elitist pressure and accelerate convergence, the worst solution is immediately overwritten by a copy of X^* , anchoring the swarm's search around the most promising known region.

Because the BPP is a combinatorial problem, the continuous position update operations of the original FDO are adapted into discrete item transfers. Following Abdul-Minaam et al. [29], *pace* is defined as a sequence of valid item transfers, each described by the tuple (item, current bin, target bin). Additionally, as the negative movement direction of the original formulation has no natural equivalent in a discrete combinatorial domain, only the positive direction is retained. The composition of the *pace* sequence is determined by a branching logic based on fw :

1. Random Exploration ($fw = 0$ or $fw = 1$): A random threshold $r \in [0, 1]$ is generated, and a proportion of items equal to r is randomly selected. Each selected item is transferred to a randomly chosen bin distinct from its current one.
2. Directed Exploitation ($0 < fw < 1$): The value of fw itself is used as the threshold proportion, selecting $fw \times n$ items. Rather than moving them randomly, each selected item is directed toward the specific bin it occupies in the global best solution X^* .

During the construction of the *pace* sequence, every proposed transfer is evaluated against the capacity constraints of the target bin. Invalid moves are rejected. Once the sequence is finalised, the moves are executed, the solution's fitness is updated incrementally, and any resultant empty bins are purged from the chromosome.

4 Experimental Setup and Methodology

4.1 Instance Generation

To evaluate the scaling behaviour, computational efficiency, and solution quality of our implemented metaheuristics, we generated custom datasets inspired by the randomly generated classes established by Delorme, Iori, and Martello [30].

Instance sets of sizes $n \in \{100, 200, 400, 600, 800, 1000, 1200, 1400, 1600\}$ were generated to observe performance trajectories from small-scale to large-scale problems. Item weights were bounded relative to the bin capacity $C = 150$: a minimum weight threshold was uniformly sampled from $[0.1C, 0.3C]$ and a maximum threshold from $[0.5C, 0.7C]$. The upper bound prevents the generation of excessively large items that cannot share a bin with any other item, which would trivialise the packing process. Item weights were then sampled from a uniform distribution bounded by these instance-specific thresholds and rounded to the nearest integer.

Because the true optimal solutions for these custom instances are unknown, solution quality is evaluated against the L_2 lower bound used by each algorithm.

4.2 Execution Environment

All algorithms were implemented in Python 3.11.2, using native data structures to handle bin creation, item assignment, and the structural mutations inherent to each metaheuristic. Experiments were conducted on a machine equipped with an AMD Ryzen 7 5825U CPU (1.6GHz base, 4.5GHz boost) and 16GB of RAM, running Debian GNU/Linux 12 (Bookworm).

4.3 Evaluation Metrics and Testing Procedure

Since the implemented metaheuristics are stochastic, their performance can fluctuate between executions. To ensure statistical reliability and capture true average-case behaviour, every algorithm was executed 5 independent times per instance. Performance is reported using two metrics.

Execution Time (s) is the mean wall-clock time in seconds across the 5 independent runs for a given instance, providing an empirical measure of computational efficiency and runtime scaling behaviour.

Percentage Difference (dif%) quantifies average-case solution quality relative to the theoretical lower bound. For a given instance i , the average number of bins used across $R = 5$ runs is:

$$\bar{S}_i = \frac{1}{R} \sum_{r=1}^R S_{i,r} \quad (1)$$

The percentage difference for that instance is then:

$$\text{Dif}_i = \left(\frac{\bar{S}_i}{L_{2,i}} - 1 \right) \times 100 \quad (2)$$

To evaluate scaling behaviour, Dif_i values are averaged across all instances within each size class. This aggregated metric provides a robust, standardised measure of how consistently each algorithm approximates the optimal solution across varying problem sizes.

5 Evaluation of Human-Crafted Metaheuristics

To establish the relative capabilities of the human-crafted metaheuristics, all eight implementations were evaluated across the generated custom datasets. These results characterise each algorithm’s baseline performance across varying instance sizes.

Table 1 details the average execution time in seconds and the percentage difference ($\text{dif}\%$) from the theoretical lower bound for each algorithm as the instance size (n) scales from 100 to 1600 items. The algorithms are ordered from left to right by overall average solution quality.

Table 1: Performance of Human-Crafted Metaheuristics: Execution Time (s) and Percentage Difference ($\text{dif}\%$) from the L_2 lower bound, ordered by overall average solution quality.

n	GA		VNS		SA		TS		GRASP		ACO		FDO		RBF	
	Time	dif%	Time	dif%	Time	dif%	Time	dif%	Time	dif%	Time	dif%	Time	dif%	Time	dif%
100	1.05	0.90	0.22	1.30	8.82	0.83	12.38	2.02	12.51	2.01	0.54	3.25	1.28	5.14	0.01	8.11
200	2.81	0.89	0.91	1.18	12.11	0.73	16.94	2.02	16.22	2.33	2.11	3.24	3.29	5.67	0.02	7.45
400	7.88	1.93	6.11	2.29	16.12	2.02	18.41	2.87	18.15	3.28	7.89	3.90	9.19	7.10	0.05	8.23
600	16.42	2.25	20.13	2.49	17.16	2.48	19.70	3.11	19.61	3.55	17.64	3.84	19.38	7.41	0.11	8.37
800	35.85	1.71	37.92	1.92	17.41	2.04	22.06	2.75	21.80	3.18	30.65	3.40	30.55	7.15	0.19	7.76
1000	63.64	1.31	48.50	1.35	20.19	1.62	22.97	2.21	22.98	2.57	48.16	2.83	44.75	6.62	0.28	7.11
1200	83.30	2.03	146.20	2.26	18.92	2.57	25.12	3.18	25.21	3.72	68.92	3.73	63.05	7.51	0.41	8.00
1400	107.93	1.48	151.84	1.62	17.58	1.93	25.79	2.77	26.74	3.22	97.27	3.27	83.91	7.10	0.53	7.43
1600	153.85	1.26	134.06	1.01	19.83	1.65	28.39	2.11	28.86	2.44	123.40	2.54	107.91	6.88	0.72	7.19
Avg dif%		1.53		1.71		1.76		2.56		2.92		3.33		6.73		7.74

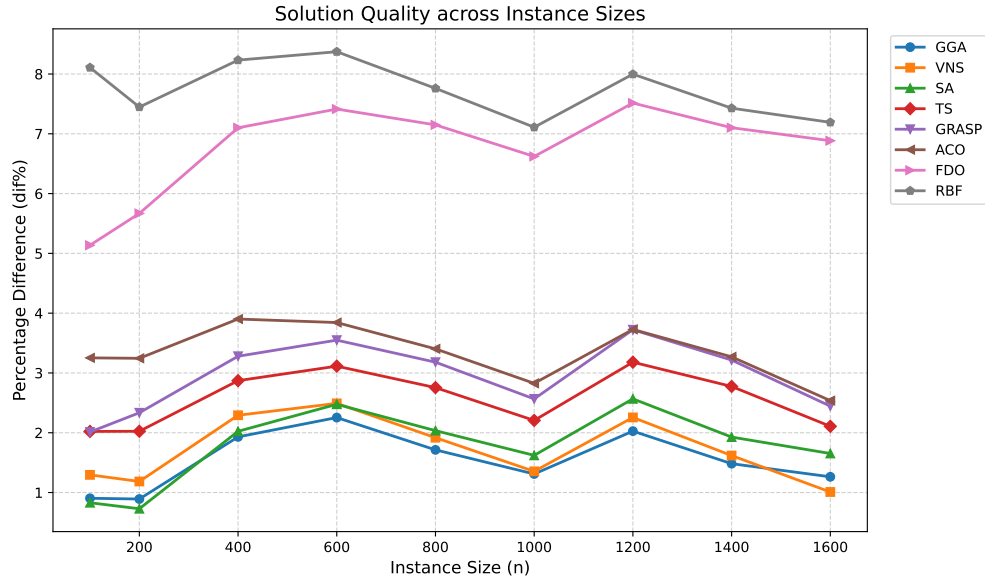


Figure 1: Average percentage difference ($\text{dif}\%$) from the theoretical lower bound across varying instance sizes.

GGA achieves the strongest overall solution quality, winning on 6 of the 9 evaluated instance sizes with deviations ranging from 0.89% to 2.25%. VNS and SA rank second and third overall by a narrow margin, though their relative performance is scale-dependent: SA outperforms VNS at smaller instance sizes ($n \leq 600$), while VNS pulls ahead at larger scales, achieving the strongest result of any algorithm at $n = 1600$ with a deviation of just 1.01%. SA’s strong small-scale performance reflects its temperature-driven search, where the fixed cooling schedule allows the algorithm sufficient time to escape local optima and converge on near-optimal solutions. TS and GRASP form a consistent mid-tier, generally producing deviations in the range of 2–4%. ACO produces broadly comparable deviations to TS and GRASP but with steeper runtime scaling, discussed below. FDO and RBF are consistently the weakest performers. RBF exceeds 7% throughout, as expected from a single-pass heuristic with no improvement phase. FDO’s poor results are likely attributable to the inherent difficulty of translating the continuous FDO velocity mechanism effectively to a discrete combinatorial domain.

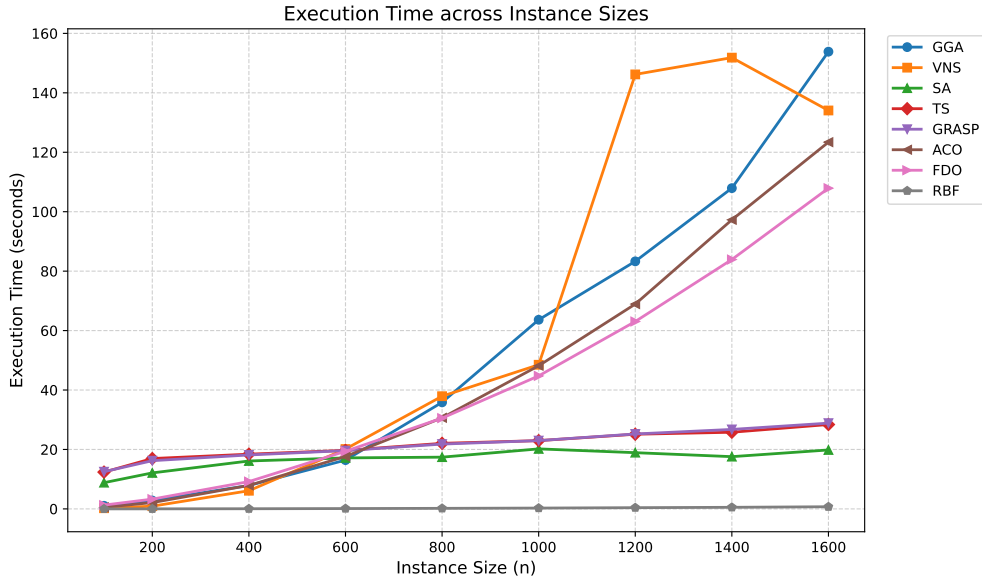


Figure 2: Average execution time in seconds across varying instance sizes.

The execution time profiles reveal an equally important dimension of algorithmic performance. SA’s execution time remains remarkably flat, hovering reliably between 8 and 20 seconds regardless of instance size, a direct consequence of its temperature-driven termination criterion: because the cooling schedule parameters are fixed constants, the number of iterations executed is effectively independent of instance size. TS and GRASP scale more gently than the population-based methods, as their fixed iteration budgets directly bound their runtime; the modest growth observed reflects slightly more expensive bin operations as average bin occupancy increases with instance size. GGA’s scaling is steep, driven by its rearrangement by pairs repair heuristic, which must process all free items against all active bins following every crossover and mutation operation. ACO scales similarly, as each ant must place more items per solution and pheromone updates must process more item pairings. FDO’s scaling mirrors GGA and ACO, driven primarily by the cost of deep-copying solution structures during each iteration. VNS scales aggressively at larger instance sizes,

driven by its exhaustive best-improvement local search, which evaluates every possible item transfer and swap across the full solution space at each iteration. The drop at $n = 1600$ likely reflects variance in instance difficulty rather than any systematic algorithmic change. RBF remains near-instantaneous throughout, though this speed advantage is entirely offset by its poor solution quality.

Among the evaluated algorithms, GGA achieves the strongest solution quality overall but incurs the highest computational cost at scale, reaching 154 seconds at $n = 1600$. VNS produces competitive solution quality, particularly at larger scales, but exhibits similarly steep runtime growth. SA offers the most favourable balance of quality and efficiency, remaining closely competitive across all instance sizes while maintaining a near-flat execution time profile. These results establish the relative capability of each human-crafted implementation, providing the baseline against which the LLM-generated variants are directly compared in Section 7.

6 LLM Algorithm Generation Methodology

6.1 Model Selection

To accurately assess the current frontier of automated algorithm design, we selected two state-of-the-art Large Language Models (LLMs): OpenAI’s GPT-5.2 and Anthropic’s Claude Opus 4.6. At the time of testing (February 2026), these represented the flagship models from their respective organisations and are widely regarded as industry leaders for code generation, consistently ranking at the top of comprehensive leaderboards such as SWE-bench [31]. While focusing on proprietary flagship models obscures the exact composition of their training data — unlike open-source alternatives such as StarCoder2 [32] — it aligns with our primary goal of evaluating the current practical capability of AI tools for operations research tasks.

We initially planned to include Google’s Gemini 3 Pro in our evaluation suite. However, because its highest-tier code generation capabilities inherently enforce continuous reasoning tokens that drastically increase API costs, conducting the full iterative testing pipeline across all algorithms became financially prohibitive, and it was subsequently excluded from the final comparison.

6.2 Prompt Engineering Strategy

All interactions with the selected LLMs were conducted programmatically via their respective APIs. While recent models exhibit less sensitivity to prompt phrasing [33], systematic prompt engineering remains important.

We employed a zero-shot prompting strategy, meaning no prior examples of Bin Packing metaheuristics were provided. This simulates the most natural human-computer interaction pattern and avoids biasing the LLM toward specific implementation styles. Furthermore, literature suggests that zero-shot prompting can sometimes outperform few-shot prompting in advanced models, as the LLM focuses purely on the task constraints rather than overfitting to the narrative style of the provided examples [34].

To maximise reasoning quality, we integrated Chain-of-Thought (CoT) prompting by append-

ing the phrase “Let’s think step by step” to the instructions [35]. We utilise this technique because CoT has been shown to yield clear improvements over direct prompting specifically for code generation tasks [36]. It is worth noting that while Li et al. [36] also introduce a more advanced structured approach to CoT, that specific method requires providing input-output examples and explicit test requirements, making it inapplicable to our zero-shot pipeline. Therefore, we rely on the baseline CoT prompt.

Additionally, we utilised persona adoption, instructing the LLM to act as a “senior algorithm designer”. While role-playing has been shown to potentially improve reasoning ability [37], the evidence for its effect on code generation specifically is limited. Since there is also no evidence suggesting it degrades performance [38], we included it as a standard practice. Furthermore, to aid the models in arriving at correct executables, the required Python function signature was provided in the prompt. This has been demonstrated to decrease the probability of encountering `ImportErrors` and `TypeErrors` in generated code [38]. Finally, the generation temperature for both models was left at their default values, as temperature primarily correlates with incoherence rather than algorithmic novelty [39].

6.3 Iterative Generation and Refinement Pipeline

To move beyond naive single-prompt generation, we implemented an automated iterative refinement pipeline inspired by the PerfCodeGen framework [40]. For every target algorithm, each LLM was allocated $k = 3$ independent candidate runs. This number was chosen to balance sample diversity against prompt budget: fewer runs would provide too small a sample of candidate implementations, while more runs would leave each individual run with an extremely limited refinement budget within the fixed prompt allocation. To manage API costs, each run has a prompt budget of $p = 5$.

The pipeline for a single candidate run operates in two distinct phases:

1. **Correctness Loop:** The initial zero-shot prompt requests the complete Python algorithm. To ensure a fair comparison and maintain reproducibility, we utilised a standardised prompt template across all LLMs, dynamically substituting only the target algorithm’s name into the prompt. The generated code is extracted and executed against a validation suite. If the code throws an execution error or violates constraints (e.g., returning invalid data types), the error traceback is fed back to the LLM in a subsequent prompt with instructions to fix the bug. The algorithm must also abide by a provided time limit (in seconds), with time management handled internally within the implementation itself rather than by external termination. If the execution exceeds this allocated time, the generation is classed as incorrect, and a timeout error is fed back to the LLM for correction. This loop repeats until the code executes successfully within all constraints or the prompt budget p is exhausted.
2. **Performance Loop:** Once a syntactically correct and executable algorithm is generated, its baseline performance is evaluated on our standard parameter-tuning dataset (test-u). The algorithm is constrained by a time limit of $\frac{1}{600}$ th of a second per item. Given the composition of the dataset, this equates to a 20-second cumulative time limit across all 20 instances. To provide a slight safety measure against anomalously lucky runs without adding excessive computational overhead, this evaluation is repeated twice.

The performance metric we optimise is the Percentage Difference (dif%) from the theoretical lower bound across the instances, ensuring the selection process specifically favours the code variant that consistently approximates the optimal solution. The subsequent refinement steps depend on the remaining prompt budget. If only one prompt remains, the LLM is provided the code alongside its performance results and asked directly to improve it. If two or more prompts remain, the first refinement iteration is split across two prompts: one asking the LLM to generate a strategic refinement plan based on the performance data, and a second asking it to apply that specific plan. For all subsequent iterations in the loop, this process is condensed: we provide the latest performance feedback and ask the LLM to improve the code.

Throughout the performance loop, newly generated code is continually validated for correctness. If an optimisation attempt breaks the code, the pipeline discards the broken version and reverts to the most performant correct version for the next prompt. At the conclusion of the $k = 3$ candidate runs, the single highest-performing correct algorithm across all runs is selected as the LLM’s final submission for that specific metaheuristic.

7 Evaluation of LLM-Generated Algorithms

All algorithms are evaluated under a fixed wall-clock time budget of $\frac{1}{600}$ th of a second per item, applied independently to each instance. A time-based stopping criterion was adopted over a fixed iteration count because per-iteration computational cost varies substantially between implementations of the same algorithm — the human designed VNS uses an exhaustive best-improvement local search while LLM equivalents employ lighter-weight search procedures — and across paradigms more broadly, making iteration counts an unreliable proxy for compute consumed. By allocating identical time budgets, observed performance differences can be attributed to algorithmic quality rather than compute. Human-implemented algorithms were adapted to use the time budget—incorporating a small safety buffer to guarantee the strict limit was not exceeded—as their primary stopping criterion. LLM-generated implementations utilise the great majority if not all of it without any adaptation.

To evaluate the generative capabilities of the selected models, we tested the LLM-generated algorithms against standard benchmark instances from BPPLIB (Falkenauer U and Scholl). Because the optima are known for these datasets, the dif% metric is calculated exactly as before but uses the known optimal rather than the theoretical lower bound. The new opt. metric denotes the total number of instances where an algorithm discovered the optimal solution in at least one of its five independent runs. The weighted average dif% reported in each table is weighted by the number of instances in each dataset class. Randomised Best Fit is excluded from this comparison, as its trivial structure leaves no meaningful implementation design space — any correct implementation produces equivalent results.

To verify stability, the 5 individual runs comprising each reported average were compared separately; the per-run scores produced consistent rankings across all algorithms, with the sole exception of the Genetic Algorithm where human and Anthropic implementations produced effectively equivalent results.

Table 2: Summary of weighted average dif% from the known optimal across all algorithm variants. Bold indicates best performance per row.

Algorithm	Human	Anthropic	OpenAI
GGA	0.14	0.15	0.87
VNS	0.27	0.58	0.38
SA	0.52	0.13	0.86
TS	0.63	0.25	1.51
GRASP	0.85	0.63	0.98
ACO	1.32	1.03	0.81
FDO	2.17	0.50	0.84

Table 2 presents the weighted average dif% from the known optimal across all evaluated algorithms, providing a macro-level view of performance. Anthropic’s Claude Opus 4.6 achieved the lowest deviation on four of the seven algorithms. The human implementation produced the strongest result on GGA and VNS, while OpenAI’s GPT-5.2 achieved the best result only on ACO.

The contrast between algorithms is notable. For GGA, where the human implementation was already highly competitive at 0.14%, Anthropic came close at 0.15% while OpenAI struggled significantly at 0.87%. For VNS, the human implementation’s exhaustive local search proved difficult to match, with OpenAI achieving 0.38% and Anthropic 0.58% against the human’s 0.27%.

7.1 Genetic Algorithm (GA)

Table 3: Results obtained by Genetic Algorithm (GA) variants applied to BPP.

Class	Inst.	GGA		Anthropic-GA		OpenAI-GA	
		opt.	dif%	opt.	dif%	opt.	dif%
falkenauer-120	20	18	0.24	18	0.29	11	1.24
falkenauer-250	20	15	0.29	14	0.38	0	1.35
falkenauer-500	20	9	0.34	10	0.31	0	1.14
falkenauer-1000	20	0	1.16	7	0.26	0	1.04
scholl-10	10	8	0.36	8	0.47	0	4.46
scholl-480	480	452	0.16	434	0.22	299	1.58
scholl-720	720	672	0.08	674	0.08	592	0.32
Total	1290	1174		1165		902	
Weighted Avg			0.14		0.15		0.87

As shown in Table 3, the human implementation narrowly outperformed Anthropic on both metrics, discovering 1174 optimal solutions at a weighted average dif% of 0.14% compared to 1165 and 0.15%. OpenAI’s implementation struggled significantly, finding only 902 optimal solutions with a weighted average dif% of 0.87%.

The three implementations differ substantially in their choice of representation. The human uses a grouping genetic algorithm with bin-level encoding throughout: crossover operates directly on bin groups, and a repair heuristic handles any items left unassigned after recombination. Anthropic uses permutation encoding decoded via Best Fit, with a crossover operator that partially preserves bin structure by alternately inheriting the fullest bins from each parent, and bin-aware mutations that attempt to empty underutilised bins. OpenAI uses continuous random-key encoding decoded to an item ordering, the most indirect representation of the three, with crossover operators that blend key vectors numerically or combine parent orderings by copying a segment from one parent and filling the remainder from the other.

The human’s group-level encoding appears to provide an advantage across most instance classes. Anthropic recovers on the larger falkenauer-1000 class, where its bin-aware mutations—which explicitly attempt to empty underutilised bins—appear to compensate for the less direct encoding. OpenAI’s poor performance is consistent with the known difficulty of indirect encodings on grouping problems—without operators that explicitly preserve or exploit bin structure, the search struggles to discover and maintain high-quality groupings.

7.2 Variable Neighbourhood Search (VNS)

Table 4: Results obtained by Variable Neighbourhood Search (VNS) variants applied to BPP.

Class	Inst.	VNS		Anthropic-VNS		OpenAI-VNS	
		opt.	dif%	opt.	dif%	opt.	dif%
falkenauer-120	20	20	0.10	10	1.03	17	0.63
falkenauer-250	20	16	0.28	0	1.23	9	0.78
falkenauer-500	20	16	0.15	0	1.03	0	0.75
falkenauer-1000	20	13	0.15	0	0.81	0	0.85
scholl-10	10	8	0.54	0	2.60	8	0.57
scholl-480	480	408	0.62	335	0.91	406	0.61
scholl-720	720	700	0.04	592	0.28	632	0.18
Total	1290	1181		937		1072	
Weighted Avg			0.27		0.58		0.38

Table 4 demonstrates that the human implementation outperformed both LLM implementations, discovering 1181 optimal solutions at a weighted average dif% of 0.27%, compared to 1072 and 0.38% for OpenAI and 937 and 0.58% for Anthropic.

The primary difference lies in the local search strategy. The human implementation uses an exhaustive best-improvement procedure that evaluates all possible item transfers and swaps across the full solution space at each iteration. Both LLM implementations use lighter-weight local searches, which likely explains the performance gap. OpenAI’s local search additionally attempts to empty bins by relocating pairs of items simultaneously, a direct bin-elimination mechanism that Anthropic’s local search lacks, which is the most plausible explanation for OpenAI’s stronger performance relative to Anthropic.

7.3 Simulated Annealing (SA)

Table 5: Results obtained by Simulated Annealing (SA) variants applied to BPP.

Class	Inst.	SA		Anthropic-SA		OpenAI-SA	
		opt.	dif%	opt.	dif%	opt.	dif%
falkenauer-120	20	6	1.44	20	0.16	6	1.44
falkenauer-250	20	0	1.48	12	0.46	0	1.47
falkenauer-500	20	0	1.31	3	0.56	0	1.32
falkenauer-1000	20	0	0.67	0	0.73	0	1.18
scholl-10	10	8	0.36	8	0.36	0	3.70
scholl-480	480	414	0.52	455	0.12	310	1.32
scholl-720	720	548	0.44	650	0.10	561	0.45
Total	1290	976		1148		877	
Weighted Avg			0.52		0.13		0.86

As detailed in Table 5, Anthropic’s Claude Opus 4.6 substantially outperformed the human implementation, discovering 1148 optimal solutions compared to 976 and reducing the weighted average dif% from 0.52% to 0.13%. The key implementation difference is the cooling schedule. Rather than using a fixed α , the generated SA estimates the number of available iterations from the remaining time budget and computes α dynamically, then recalibrates the temperature against wall-clock progress every 500 iterations. This makes the search more responsive to the available time budget, allowing it to cool more appropriately across different instance sizes.

GPT-5.2 underperformed the human implementation, yielding only 877 optimal solutions and a weighted average dif% of 0.86%. The root cause is a fundamental flaw in the objective function. OpenAI’s implementation scores solutions as $\text{bins_used} \times M + \text{total_slack} \times 0.01$, where $M = C \times (n + 5)$. The $\text{bins_used} \times M$ term only changes when a bin is eliminated entirely. The $\text{total_slack} \times 0.01$ term is conserved across all feasible moves: moving an item from one bin to another gains exactly as much slack in the source bin as it loses in the destination. Consequently, the objective function returns an identical value for the vast majority of moves and fails to guide the search.

7.4 Tabu Search (TS)

Table 6: Results obtained by Tabu Search (TS) variants applied to BPP.

Class	Inst.	TS		Anthropic-TS		OpenAI-TS	
		opt.	dif%	opt.	dif%	opt.	dif%
falkenauer-120	20	7	1.42	15	0.52	6	1.44
falkenauer-250	20	0	1.42	8	0.69	0	1.48
falkenauer-500	20	0	1.30	2	0.68	0	1.34
falkenauer-1000	20	0	1.15	0	0.64	0	1.21
scholl-10	10	7	1.11	5	3.60	0	6.06
scholl-480	480	364	0.88	425	0.31	236	3.01
scholl-720	720	582	0.37	646	0.13	547	0.47
Total	1290	960		1101		789	
Weighted Avg			0.63		0.25		1.51

As seen in Table 6, Anthropic’s Claude Opus 4.6 substantially outperformed both the human implementation and OpenAI’s, discovering 1101 optimal solutions compared to 960 and 789 respectively, and reducing the weighted average dif% from 0.63% to 0.25%. In contrast, OpenAI’s GPT-5.2 underperformed the human implementation on nearly all instance classes and performed particularly poorly on the scholl-10 and scholl-480 sets.

The most notable structural difference between the implementations is Anthropic’s adoption of an iterative bin-reduction strategy. While the human and OpenAI implementations perform conventional feasibility-preserving tabu search starting from a valid packing, the Anthropic implementation repeatedly attempts to reduce the number of bins by constructing an intentionally infeasible packing and then applying tabu search to minimise overflow until feasibility is restored.

The human and OpenAI implementations differ from each other primarily in their bin-targeting strategy: the human concentrates around 60% of relocate moves on the single emptiest bin, creating sustained pressure toward eliminating it, whereas OpenAI distributes attention across the lightest third of bins, which is less likely to fully empty any individual bin.

7.5 Greedy Randomised Adaptive Search Procedure (GRASP)

Table 7: Results obtained by GRASP variants applied to BPP.

Class	Inst.	GRASP		Anthropic-GRASP		OpenAI-GRASP	
		opt.	dif%	opt.	dif%	opt.	dif%
falkenauer-120	20	4	1.70	16	0.51	14	0.74
falkenauer-250	20	0	1.51	6	0.90	5	0.99
falkenauer-500	20	0	1.40	0	1.10	0	1.06
falkenauer-1000	20	0	1.31	0	1.09	0	1.04
scholl-10	10	2	2.03	0	4.74	0	4.82
scholl-480	480	323	1.29	366	1.17	299	2.04
scholl-720	720	556	0.47	629	0.17	620	0.22
Total	1290	885		1017		938	
Weighted Avg			0.85		0.63		0.98

As detailed in Table 7, Anthropic’s implementation outperformed both the human implementation and OpenAI, discovering 1017 optimal solutions compared to 885 and 938 respectively, and reducing the weighted average dif% from 0.85% to 0.63%. The key structural difference from the human implementation is in the local search: Anthropic uses a greedy bin-emptying procedure that repeatedly attempts to relocate all items from the lightest bin via best-fit, falling back to single-item swap moves between bins when direct emptying fails, whereas the human implementation uses tabu search as its local search phase. Anthropic performs worse than the human only on scholl-10, where tabu-based search appears better suited to the small instance size.

OpenAI discovered more optimal solutions than the human implementation (938 vs 885) but registered a higher weighted average dif% (0.98% vs 0.85%), and underperformed Anthropic on both metrics. Its local search employs a broader range of neighbourhood operators than the other two implementations, yet this does not translate to stronger average-case performance. The most notable result is scholl-480, where OpenAI performs substantially worse than both other implementations.

7.6 Ant Colony Optimisation (ACO)

Table 8: Results obtained by Ant Colony Optimisation (ACO) variants applied to BPP.

Class	Inst.	ACO		Anthropic-ACO		OpenAI-ACO	
		opt.	dif%	opt.	dif%	opt.	dif%
falkenauer-120	20	4	1.90	15	0.86	14	0.88
falkenauer-250	20	0	1.86	2	1.06	3	1.04
falkenauer-500	20	0	1.66	0	1.04	0	0.89
falkenauer-1000	20	0	1.57	0	1.07	0	0.89
scholl-10	10	0	5.99	0	5.06	0	4.92
scholl-480	480	269	2.19	289	2.18	319	1.59
scholl-720	720	532	0.63	619	0.21	623	0.21
Total	1290	805		925		959	
Weighted Avg			1.32		1.03		0.81

As shown in Table 8, both LLM implementations outperformed the human implementation, which discovered 805 optimal solutions with a weighted average dif% of 1.32%. OpenAI achieved the strongest overall result, finding 959 optimal solutions and a weighted average dif% of 0.81%, with Anthropic close behind at 925 and 1.03%.

ACO is sufficiently computationally expensive that all implementations complete very few iterations within the time budget, leaving pheromone trail differentiation from initial values minimal. Performance is therefore driven primarily by single-construction quality rather than accumulated pheromone learning, meaning the results reflect early-stage behaviour rather than the algorithm operating as intended.

Since construction quality is the dominant factor, the most plausible explanation for OpenAI’s stronger performance lies in two aspects of its construction. First, it includes a look-ahead that rewards placing an item in a bin if the remaining capacity afterwards is exactly fillable by another unpacked item, directly targeting tight bin filling at each step. Second, it evaluates more ants per iteration than Anthropic across all instance sizes, producing greater construction diversity within the available time budget.

7.7 Fitness Dependent Optimizer (FDO)

Table 9: Results obtained by Fitness Dependent Optimizer (FDO) variants applied to BPP.

Class	Inst.	FDO		Anthropic-FDO		OpenAI-FDO	
		opt.	dif%	opt.	dif%	opt.	dif%
falkenauer-120	20	0	3.45	15	0.72	8	1.36
falkenauer-250	20	0	3.72	6	0.86	0	1.32
falkenauer-500	20	0	4.36	0	0.89	0	1.28
falkenauer-1000	20	0	4.57	0	0.64	0	1.21
scholl-10	10	0	6.66	0	2.71	0	5.49
scholl-480	480	252	2.26	361	0.88	309	1.45
scholl-720	720	335	1.84	639	0.18	590	0.32
Total	1290	587		1021		907	
Weighted Avg			2.17		0.50		0.84

Table 9 illustrates that the human implementation struggled considerably with FDO, discovering only 587 optimal solutions and recording the highest weighted average dif% of any algorithm in this study at 2.17%. The core difficulty lies in translating FDO’s continuous velocity concept to a discrete domain: the human implementation maps velocity directly to item transfers between bins, discarding any infeasible moves.

Neither LLM produced a faithful FDO translation. Anthropic computes a fitness weight but replaces the velocity mechanic entirely with permutation-based evolutionary operators. OpenAI applies a continuous velocity-style update to a real-valued encoding that is decoded into a packing, which preserves the spirit of the velocity concept but departs substantially from the original FDO framework. For both models, superior performance likely reflects the suitability of their chosen approaches for the BPP rather than the quality of the FDO translation. The human implementation is therefore the only meaningful FDO comparison point, against which both LLM implementations comfortably outperform — Anthropic discovering 1021 optimal solutions at 0.50% and OpenAI 907 at 0.84%.

8 Discussion

8.1 Human-Implemented Algorithms

The results from Section 5 provide a clear answer to the first research question. GGA, VNS, and SA form a strong top tier, with average deviations of 1.53%, 1.71%, and 1.76% respectively, though their relative performance is scale-dependent as discussed in Section 5. SA offers the most favourable balance of quality and computational efficiency given its near-flat runtime profile. TS, GRASP, and ACO form a consistent mid-tier, with ACO exhibiting steeper runtime scaling than the other two. FDO and RBF are consistently the weakest performers. RBF’s poor performance is expected given its single-pass nature, while FDO’s weakness is likely attributable to the inherent difficulty of adapting its continuous velocity mechanism to a discrete combinatorial domain, rather than any computational constraint.

8.2 LLM-Generated Algorithms

The results in Section 7 provide a broadly positive answer to the second research question. Both LLMs produced functionally correct code for six of the seven algorithms, deviating from the specified paradigm only on FDO, and both frequently matched or exceeded human-level performance. Anthropic’s Claude Opus 4.6 achieved the strongest result on four of the seven algorithms, with the human implementation outperforming both LLMs on GGA and VNS, and OpenAI achieving the best result on ACO. The Anthropic SA implementation is especially notable: rather than using a fixed cooling parameter, it adopts a dynamic schedule that adapts to the available time budget, producing a more effective implementation than the human’s fixed-parameter approach. Similarly, the Anthropic TS implementation uses an iterative bin-reduction formulation rather than conventional feasibility-preserving search, which proved substantially more effective on the benchmark instances. In both cases the human implementation did not employ these approaches, and the performance gap reflects this difference in design.

However, the results also reveal clear limitations. OpenAI’s SA implementation contained a fundamental flaw in its objective function that rendered the search almost entirely uninformative, and its TS implementation underperformed the human implementation across nearly all instance classes, most plausibly due to its bin-targeting strategy distributing attention across the lightest third of bins rather than focusing sustained pressure on a single emptiest bin. These failures were not syntactic — the code executed correctly and produced valid packings — but reflect deeper reasoning errors in algorithm design. This distinction is important: correctness and performance are separable, and the iterative pipeline’s correctness loop cannot detect the latter.

8.3 Patterns Across LLMs

Addressing the third research question, the two models exhibit distinct strengths and weaknesses. Anthropic performs more consistently across the full suite, producing competitive implementations for every algorithm. OpenAI shows higher variance: it achieves the strongest result on ACO, where construction-level decisions appear to play to its strengths, but struggles considerably on SA where understanding of the objective function and search guidance is critical.

Both models deviated from the specified paradigm on FDO, with neither producing a faithful velocity-based translation. This is perhaps unsurprising given that FDO is a relatively recent and more sparsely documented algorithm whose discrete adaptation for the BPP has limited prior work. In this context, the models likely defaulted to more familiar and well-represented approaches rather than attempting a genuine FDO translation. Whether this constitutes a failure depends on how the task is framed: if the goal is to produce the best-performing algorithm, paradigm deviation may be acceptable or even desirable; if the goal is to evaluate a specific algorithm’s generative capability, it undermines the comparison.

8.4 Limitations

Several limitations of this study warrant acknowledgement. First, the human baselines benefited from manual parameter tuning without a fixed budget, whereas the LLMs were constrained to a prompt budget of $p = 5$ per run. Furthermore, during the refinement phase, the models only received the most recent working code and its performance metrics, lacking visibility into previously attempted or discarded approaches. The human implementations also prioritised breadth over depth, omitting certain techniques from the reference literature (e.g., the Minimum Bin Slack heuristic in VNS) that could have improved individual algorithm performance. Second, only two LLMs were evaluated. The planned inclusion of Google’s Gemini 3 Pro was excluded on cost grounds, and reasoning-augmented model variants were similarly excluded, both of which restrict the generalisability of conclusions about inter-model differences and may have produced meaningfully different results, particularly for algorithms where subtle design choices have an outsized impact on performance. Third, all implementations were written in Python, which imposes computational constraints that disproportionately affect algorithms such as ACO and FDO. A compiled language implementation might alter the relative rankings of these algorithms. Fourth, the BPP is a well-studied problem with a substantial body of published implementations, meaning the models may have had direct or indirect exposure to relevant design patterns during training. This makes it difficult to distinguish genuine algorithmic reasoning from retrieval of memorised solutions, and strong LLM performance should be interpreted with this caveat in mind. Fifth, the LLM generation pipeline is itself stochastic — running it again would likely produce different implementations, and the results reflect one particular instantiation of each model’s generative capability. Finally, the initial prompts expressed a preference for fixed-iteration implementations with a time budget as a secondary stopping criterion, rather than specifying time-based termination directly; many of the generated implementations nonetheless adopted time-based stopping as their primary criterion, and a cleaner design would have specified this from the outset to ensure consistency across all implementations.

8.5 Future Work

Several directions present themselves as natural extensions of this study. The most direct way to address the training data contamination concern would be to evaluate LLM-generated metaheuristics on less well-studied combinatorial problems, where memorised design patterns are less likely to be available, providing a stronger test of genuine algorithmic reasoning ability. A complementary direction would be to evaluate on more structurally complex problems such as aircrew scheduling, where constructing feasible solutions is itself non-trivial and requires nuanced design decisions — including how to handle infeasible solutions during the search, since strongly infeasible solutions can carry useful structural information that guides the algorithm toward feasibility. Including a broader range of LLMs, particularly open-source models whose training data is known, would also help disentangle knowledge retrieval from reasoning capability. Evaluating reasoning-augmented model variants would test whether such capabilities meaningfully improve metaheuristic design quality, particularly for algorithms where the human baselines currently outperform LLM implementations. Additionally, enriching the iterative refinement pipeline—for instance, by supplying the LLM with a complete history of successful and discarded approaches rather than just the latest working version—could yield stronger implementations by allowing the model to learn from its own failed attempts.

Finally, all implementations in this study were written in Python, which is a relatively accessible language to implement in. It would be interesting to evaluate whether the conclusions hold when implementations are required in a lower-level compiled language, where the implementation complexity is higher and the gap between a correct and a performant solution is wider.

9 Conclusion

This study evaluated a diverse suite of human-implemented randomised algorithms for the 1D Bin Packing Problem and compared them against zero-shot implementations generated by two state-of-the-art LLMs. Across all three research questions, the results yield clear and practically meaningful findings.

Among the human-implemented algorithms, GGA, VNS, and SA formed a strong top tier, with GGA achieving the strongest solution quality overall and SA offering the most favourable balance of quality and computational efficiency. TS, GRASP, and ACO formed a consistent mid-tier, while FDO and RBF were consistently the weakest performers.

The LLM evaluation produced results that are both impressive and instructive. Anthropic’s Claude Opus 4.6 achieved the strongest result on four of the seven algorithms, with the human implementation outperforming both LLMs only on GGA and VNS. Several Anthropic implementations incorporated design choices the human baseline did not — the dynamic cooling schedule in its SA implementation and the iterative bin-reduction framework in its TS implementation are both well-established techniques that the human implementation did not employ, and their inclusion contributed meaningfully to the performance advantage. OpenAI’s GPT-5.2 showed higher variance, achieving the best result on ACO while struggling on TS and SA. The SA objective function flaw illustrates a category of failure that is particularly difficult to detect automatically: the code is correct, the search runs, and valid packings are produced, but the algorithm is not actually doing what it claims to.

Neither model produced a faithful FDO translation, most plausibly because FDO is a recent and sparsely documented algorithm with limited prior work on discrete adaptations. In this case, both models defaulted to more familiar approaches, and their superior performance over the human FDO implementation likely reflects the suitability of those approaches for the BPP rather than the quality of the FDO translation itself.

Overall, the findings suggest that flagship LLMs are now capable of producing competitive metaheuristic implementations from scratch for well-studied combinatorial problems, drawing on a broad knowledge of existing techniques to assemble implementations that compare favourably with carefully tuned human baselines. However, they remain capable of subtle but consequential reasoning errors that pass all correctness checks while undermining the search. For practitioners, this implies that LLM-generated optimisation code should be treated as a strong starting point requiring algorithmic review, rather than a finished product.

References

- [1] Maxence Delorme, Manuel Iori, Silvano Martello, Bin packing and cutting stock problems: Mathematical models and exact algorithms, *European Journal of Operational Research*, Volume 255, Issue 1, 2016, Pages 1-20, ISSN 0377-2217, <https://doi.org/10.1016/j.ejor.2016.04.030>.
- [2] Munien, Chanaleä and Ezugwu, Absalom E.. "Metaheuristic algorithms for one-dimensional bin-packing problems: A survey of recent advances and applications" *Journal of Intelligent Systems*, vol. 30, no. 1, 2021, pp. 636-663. <https://doi.org/10.1515/jisys-2020-0117>
- [3] Garey, Michael R. and Johnson, David S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [4] Sean Luke, 2013, *Essentials of Metaheuristics*, Lulu, second edition, available for free at <https://people.cs.gmu.edu/~sean/book/metaheuristics/Essentials.pdf>
- [5] Yang, X.-S. (2011). Metaheuristic Optimization. *Scholarpedia*, 6 (8), 11472. <https://doi.org/10.4249/scholarpedia.11472>
- [6] Kucharavy, A. (2024). Fundamental limitations of generative LLMs. In *Large Language Models in Cybersecurity: Threats, Exposure and Mitigation* (pp. 55–64). Springer Nature Switzerland, Cham. <https://library.oapen.org/bitstream/handle/20.500.12657/90897/978-3-031-54827-7.pdf>
- [7] Dósa, G. (2007). The tight bound of First Fit Decreasing bin-packing algorithm is $\text{FFD(I)} \leq 11/9 \text{ OPT(I)} + 6/9$. In B. Chen, M. Paterson, & G. Zhang (Eds.), *Combinatorics, Algorithms, Probabilistic and Experimental Methodologies (ESCAPE 2007)* (Lecture Notes in Computer Science, vol. 4614, pp. 1–11). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-74450-4_1
- [8] Sonuç, E., Şen, B. and Bayır, Ş., 2017. Solving Bin Packing Problem Using Simulated Annealing. In: *Proceedings of the 65th ISERD International Conference*, Mecca, Saudi Arabia, 23–24 January 2017. World Research Library, pp. 83–85. https://worldresearchlibrary.org/up_proc/pdf/635-148739346283-85.pdf
- [9] Krzysztof Fleszar, Khalil S. Hindi, New heuristics for one-dimensional bin-packing, *Computers & Operations Research*, Volume 29, Issue 7, 2002, Pages 821-839, ISSN 0305-0548, [https://doi.org/10.1016/S0305-0548\(00\)00082-4](https://doi.org/10.1016/S0305-0548(00)00082-4).
- [10] Falkenauer, E. A hybrid grouping genetic algorithm for bin packing. *Journal of Heuristics*, 2(1):5–30, 1996. <https://doi.org/10.1007/BF00226291>
- [11] M. Quiroz-Castellanos, L. Cruz-Reyes, J. Torres-Jimenez, C. Gómez S., H. J. Fraire Huacuja, and A. C. F. Alvim. A grouping genetic algorithm with controlled gene transmission for the bin packing problem. *Computers & Operations Research*, 55:52-64, 2015. <https://doi.org/10.1016/j.cor.2014.10.010>
- [12] Levine, J., & Ducatelle, F. (2004). Ant colony optimization and local search for bin packing and cutting stock problems. *Journal of the Operational Research Society*, 55(7), 705–716. <https://doi.org/10.1057/palgrave.jors.2601771>

- [13] Chang Y, Wang X, Wang J, Wu Y, Yang L, Zhu K, Chen H, Yi X, Wang C, Wang Y, Ye W, Zhang Y, Chang Y, Yu PS, Yang Q, Xie X. A Survey on Evaluation of Large Language Models. *ACM Trans. Intell. Syst. Technol.* 2024;15(3):Article 39 (45 pages). <https://doi.org/10.1145/3641289>
- [14] Bubeck S, Chandrasekaran V, Eldan R, Gehrke J, Horvitz E, Kamar E, Lee P, Lee YT, Li Y, Lundberg S, Nori H, Palangi H, Ribeiro MT, Zhang Y. Sparks of Artificial General Intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*. 2023. <https://arxiv.org/abs/2303.12712>
- [15] Wang, L., Shi, C., Du, S., Tao, Y., Shen, Y., Zheng, H., & Qiu, X. (2024). Performance Review on LLM for solving leetcode problems. In *2024 4th International Symposium on Artificial Intelligence and Intelligent Manufacturing (AIIM)* (pp. 1050–1054). <https://doi.org/10.1109/AIIM64537.2024.10934280>
- [16] Coignion, T., Quinton, C., & Rouvoy, R. (2024). A Performance Study of LLM-Generated Code on Leetcode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (pp. 79–89). Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3661167.3661221>
- [17] H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, and G. Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *Advances in Neural Information Processing Systems*, 2024. <https://arxiv.org/abs/2402.01145>
- [18] C. D. Rosin. Using code generation to solve open instances of combinatorial design problems. *arXiv preprint arXiv:2501.17725*, 2025. <https://arxiv.org/abs/2501.17725>
- [19] Silvano Martello, Paolo Toth, Lower bounds and reduction procedures for the bin packing problem, *Discrete Applied Mathematics*, Volume 28, Issue 1, 1990, Pages 59–70, ISSN 0166-218X, [https://doi.org/10.1016/0166-218X\(90\)90094-S](https://doi.org/10.1016/0166-218X(90)90094-S).
- [20] Fekete, S. & Schepers, J. New classes of fast lower bounds for bin packing problems. *Math. Program.* 91, 11–31 (2001). <https://doi.org/10.1007/s101070100243>
- [21] Motwani, R. and Raghavan, P. *Randomized Algorithms*. Cambridge University Press, 1995.
- [22] Gendreau, Michel and Potvin, Jean-Yves, editors. *Handbook of Metaheuristics*, Third Edition. *International Series in Operations Research & Management Science*. Springer, 2019. <https://doi.org/10.1007/978-3-319-91086-4>
- [23] Lodi, A., Martello, S. & Vigo, D. TSPack: A Unified Tabu Search Code for Multi-Dimensional Bin Packing Problems. *Ann Oper Res* 131, 203–213 (2004). <https://doi.org/10.1023/B:ANOR.0000039519.03572.08>
- [24] N. Mladenović, P. Hansen, Variable neighborhood search, *Computers & Operations Research*, Volume 24, Issue 11, 1997, Pages 1097–1100, ISSN 0305-0548, [https://doi.org/10.1016/S0305-0548\(97\)00031-2](https://doi.org/10.1016/S0305-0548(97)00031-2).
- [25] Gupta JND, Ho JC. A new heuristic algorithm for the one-dimensional bin-packing problem. *Production Planning & Control*. 1999;10(6):598–603. <https://doi.org/10.1080/095372899232894>

- [26] Stützle, T., & Hoos, H. H. (2000). MAX-MIN Ant System. *Future Generation Computer Systems*, 16 (8), 889–914. [https://doi.org/10.1016/S0167-739X\(00\)00043-1](https://doi.org/10.1016/S0167-739X(00)00043-1)
- [27] Aaron C. Zecchin, Angus R. Simpson, Holger R. Maier, Michael Leonard, Andrew J. Roberts, Matthew J. Berrisford, Application of two ant colony optimisation algorithms to water distribution system optimisation, *Mathematical and Computer Modelling*, Volume 44, Issues 5–6, 2006, Pages 451-468, ISSN 0895-7177, <https://doi.org/10.1016/j.mcm.2006.01.005>
- [28] Abdullah JM, Ahmed T. Fitness Dependent Optimizer: Inspired by the Bee Swarming Reproductive Process. *IEEE Access*. 2019;7:43473–43486. <https://doi.org/10.1109/ACCESS.2019.2907012>
- [29] Abdul-Minaam DS, Al-Mutairi WMES, Awad MA, El-Ashmawi WH. An Adaptive Fitness-Dependent Optimizer for the One-Dimensional Bin Packing Problem. *IEEE Access*. 2020;8:97959–97974. <https://doi.org/10.1109/ACCESS.2020.2985752>
- [30] M. Delorme, M. Iori, and S. Martello. BPPLIB: A library for bin packing and cutting stock problems. *Optimization Letters*, 12(2):235-250, 2018. <https://github.com/mdelorme2/BPPLIB>.
- [31] Scale AI. SWE-bench Leaderboard. 2026. https://scale.com/leaderboard/swe_bench_pro_public. [Accessed: 25-February-2026]
- [32] A. Lozhkov, R. Li, L. Ben Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, et al. StarCoder 2 and The Stack v2: The Next Generation. *arXiv preprint arXiv:2402.19173*, 2024. <https://arxiv.org/abs/2402.19173>
- [33] G. Wang, Z. Sun, S. Ye, Z. Gong, Y. Chen, Y. Zhao, Q. Liang, and D. Hao. Do advanced language models eliminate the need for prompt engineering in software engineering? *ACM Transactions on Software Engineering and Methodology*, 2025. <https://doi.org/10.1145/3771933>
- [34] L. Reynolds and K. McDonell. Prompt programming for large language models: Beyond the few-shot paradigm. *arXiv preprint arXiv:2102.07350*, 2021. <https://arxiv.org/abs/2102.07350>
- [35] T. Kojima, S. Gu, M. Reid, Y. Matsuo, and M. Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199-22213, 2022. <https://arxiv.org/abs/2205.11916>
- [36] J. Li, G. Li, Y. Li, and Z. Jin. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2), 2024. <https://doi.org/10.1145/3690635>
- [37] A. Deshpande, V. Murahari, T. Rajpurohit, A. Kalyan, and K. Narasimhan. Better zero-shot reasoning with role-play prompting. *arXiv preprint arXiv:2308.07702*, 2023. <https://arxiv.org/abs/2308.07702>
- [38] R. Khojah, F. G. de Oliveira Neto, M. Mohamad, and P. Leitner. The impact of prompt programming on function-level code generation. *arXiv preprint arXiv:2412.20545*, 2025. <https://arxiv.org/abs/2412.20545>

- [39] M. Peeperkorn, T. Kouwenhoven, D. Brown, and A. Jordanous. Is temperature the creativity parameter of large language models? arXiv preprint arXiv:2405.00492, 2024. <https://arxiv.org/abs/2405.00492>
- [40] Y. Peng, A. D. Gotmare, M. Lyu, C. Xiong, S. Savarese, and D. Sahoo. PerfCodeGen: Improving performance of LLM generated code with execution feedback. arXiv preprint arXiv:2412.03578, 2024. <https://arxiv.org/abs/2412.03578>